



# Shell Scripting

NCSC101

**Prof. Pranay Kumar Saha**

September 8, 2025





## Where We Are

- You already know: basic Unix commands, pipes, **grep**, **sed**, **awk**
- Today: turn those skills into **reusable shell scripts**



# Why Write Shell Scripts?

- Automate repetitive tasks (backups, log parsing, deployments)
- Glue different CLI tools together
- Fast to write, easy to run on servers/CI
- Great for data wrangling and ops workflows



# Shebang & Making It Executable

```
1 #!/usr/bin/env bash  
2 echo "Hello from a script!"
```

Listing 1: hello.sh

```
1 chmod +x hello.sh  
2 ./hello.sh  
3 # or: bash hello.sh
```

Listing 2: Run it

- Use `#!/usr/bin/env bash` for portability across systems



# Variables & Quoting

```
1 name="Ada"
2 echo Hello $name      # may give wrong output
3 echo "Hello $name"    # safe: preserves spaces
4 echo 'Hello $name'    # single quotes: no expansion
```

Listing 3: Assignment & quoting

- Always quote expansions: "\$var" and "\$@"



# Command Substitution & Arithmetic

```
1 today=$(date +%F)
2 echo "Today is $today"
3
4 x=7; y=5
5 sum=$(( x + y ))
6 echo "Sum: $sum"
```

Listing 4: Substitution & arithmetic



# Redirection & Pipes

```
1 cmd >out.txt 2>err.txt    # redirect stdout / stderr
2 cmd >>out.txt              # append
3 cmd <input.txt             # stdin from file
4 cmd1 | cmd2                # pipe
```

Listing 5: Redirection basics

```
1 cat <<'EOF' > note.txt
2 Line 1
3 $PATH will not expand because we used single quotes
4 EOF
```

Listing 6: Here-doc



## Exit Status & Short-Circuit

```
1 # Last command status in $?  
2 false  
3 echo "$?"      # 1  
4  
5 true && echo "OK" # runs  
6 false || echo "fallback on error" # runs
```

Listing 7: Exit status



# What is Conditional Logic?

## The Core Idea

Conditional logic allows a script to make decisions and execute different blocks of code based on whether a certain condition is true or false.

- This is the primary way to create intelligent, non-linear scripts.
- In Bash, this is primarily handled by the **if**, **elif**, and **else** keywords.
- Conditions are typically evaluated within double square brackets: `[[ ... ]]`.



# 1. The Basic 'if' Statement

For executing code only when a condition is true.

## Syntax

```
if [[ condition ]]; then  
    # commands to run if condition is true  
fi
```

## Example: Checking a username

```
username="pranay"  
if [[ "$username" == "pranay" ]]; then  
    echo "Access granted. Welcome, Pranay!"  
fi
```

Access granted. Welcome, Pranay!



## 2. The 'if-else' Statement

For choosing between two different paths.

### Syntax

```
if [[ condition ]]; then
    # commands to run if condition is true
else
    # commands to run if condition is false
fi
```

```
file_path="/etc/hosts"
if [[ -f "$file_path" ]]; then
    echo "$file_path exists."
else
    echo "$file_path does NOT exist."
fi
```



### 3. The 'if-elif-else' Statement

For handling multiple, mutually exclusive conditions.

#### Syntax

```
if [[ condition1 ]]; then
    # commands for condition1
elif [[ condition2 ]]; then
    # commands for condition2
else
    # commands if no conditions are met
fi
```



### 3. The 'if-elif-else' Statement

For handling multiple, mutually exclusive conditions.

```
score=85
if [[ "$score" -gt 90 ]]; then
    echo "Grade: A"
elif [[ "$score" -gt 80 ]]; then
    echo "Grade: B"
else
    echo "Grade: C or lower"
fi
```

Grade: B



# Common Test Conditions inside `{[ ... ]}`

The building blocks of your 'if' statements.

## String Comparisons

- `==` Equal to
- `!=` Not equal to
- `-z` Is empty
- `-n` Is not empty

## File Tests

- `-e` Exists (file or dir)
- `-f` Is a regular file
- `-d` Is a directory
- `-r` Is readable

## Numeric Comparisons

- `-eq` Equal
- `-ne` Not equal
- `-gt` Greater than
- `-ge` Greater or equal
- `-lt` Less than
- `-le` Less or equal

Always quote variables inside the brackets (e.g., `[[ -f "$file" ]]`) to prevent errors with empty variables or values containing spaces.



# What is a 'for' loop?

## The Core Idea

A **for** loop is a control flow statement for iterating over a sequence of items. It executes a set of commands once for each item in the list.

- We will explore several common patterns for creating that "sequence of items".
- Understanding these patterns is key to writing effective and versatile shell scripts.



## Tests & Conditionals

```
1 read -r file
2 if [[ -f "$file" ]]; then
3     echo "It's a file"
4 elif [[ -d "$file" ]]; then
5     echo "It's a directory"
6 else
7     echo "Not found"
8 fi
```

Listing 8: if / elif / else

- Prefer `[[ ... ]]` (bash) over `[ ... ]` to avoid many pitfalls



# 1. The C-Style Loop

For traditional, numeric iteration.

## Syntax

```
for (( initializer ; condition; incrementer)); do  
    # commands  
done
```

## Example: Counting from 1 to 3

```
for (( i=1; i<=3; i++ )); do  
    echo "Count is: $i"  
done
```

Count is: 1

Count is: 2

Count is: 3



## 2. The List-Based Loop

For iterating over a fixed set of strings.

### Syntax

```
for var in "item1" "item2" "item3"; do  
    # commands using $var  
done
```

### Example: Processing Fruits

```
for fruit in "Apple" "Banana" "Cherry Pie"; do  
    echo "Current fruit: $fruit"  
done
```

Current fruit: Apple

Current fruit: Banana

Current fruit: Cherry Pie



### 3. Looping Over Files (Globbing)

For batch processing files in a directory.

#### Syntax

```
for file in *.log; do
    # commands using "$file"
done
```

#### Example: Renaming .txt to .md

```
# Directory contains: a.txt, b.txt
for f in *.txt; do
    mv -- "$f" "${f%.txt}.md"
    echo "Renamed $f to ${f%.txt}.md"
done
```



## 5. Looping Over Brace Expansion

For simple numeric or character ranges.

### Syntax

```
for var in {START..END}; do  
    # commands using $var  
done
```

```
for i in {1..4}; do  
    echo "Number: $i"  
done
```

Number: 1  
Number: 2  
Number: 3  
Number: 4

```
for c in {a..d}; do  
    echo "Letter: $c"  
done
```

Letter: a  
Letter: b  
Letter: c  
Letter: d



# What Are Command-Line Arguments?

## The Core Idea

Arguments are the extra pieces of information you provide to a script when you run it from the terminal. They allow you to pass data into your script, making it reusable and dynamic.

## Example Command

```
./my_script.sh "input file .txt" /tmp/output
```

- Here, "input file.txt" and /tmp/output are arguments.
- The shell provides special variables to access these values from within your script.



## Key Special Variables for Arguments

The shell automatically populates these for you.

|                            |                                                                                                                                        |
|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| <code>\$0</code>           | The name of the script itself. Useful for printing usage instructions.                                                                 |
| <code>##</code>            | The <b>total count</b> of arguments provided. An integer.                                                                              |
| <code>\$1, \$2, ...</code> | The individual arguments, accessed by their position. These are called "positional parameters."                                        |
| <code>\$@</code>           | Expands to <b>all arguments</b> , with each one correctly quoted as a separate string. This is the safest way to access all arguments. |



# Key Special Variables for Arguments

The shell automatically populates these for you.

## Example Code

```
# If you run: ./script.sh hello "world !"
echo "Script name: $0" # -> ./script.sh
echo "Argument count: $# " # -> 2
echo "First argument: $1" # -> hello
echo "Second argument: $2" # -> world !
```



# Accessing Individual Arguments

Using \$1, \$2, etc.

## Important: Always Check First!

Accessing a variable like \$1 when no arguments were given can lead to errors. It's crucial to check the argument count (\$#) first.

### Example Code

```
# Check if at least two arguments exist
if [[ "$#" -ge 2 ]]; then
    echo "The first argument is: $1"
    echo "The second argument is: $2"
else
    echo "Error: Please provide at least two arguments."
fi
```



# Looping Over All Arguments

The safest method using a for loop and “\$@”

## Example Code

```
1 if [[ "$#" -eq 0 ]]; then
2     echo "No arguments to loop through."
3 else
4     echo "Looping through all arguments:"
5     count=1
6     for arg in "$@"; do
7         echo "  Arg $count: $arg"
8         ((count++))
9     done
10 fi
```

This is the standard, most robust way to process every argument passed to your script.



# Practical Example: Validating Input

A common pattern in real-world scripts.

## Example Code

```
1 EXPECTED_ARGS=2
2 if [[ "$#" -ne "$EXPECTED_ARGS" ]]; then
3     echo "Error: This script requires exactly $EXPECTED_ARGS arguments."
4     echo "Usage: $0 <source_file> <destination_directory>"
5     exit 1 # Exit with an error code
6 else
7     echo "Success: You provided the correct number of arguments."
8     echo "Source file would be: $1"
9     echo "Destination directory would be: $2"
10 fi
```



# Beyond Command-Line Arguments

## Two Main Ways to Get Input

1. **Command-Line Arguments:** Input is provided once when the script starts. It is non-interactive.
2. **Interactive Input:** The script pauses during execution to ask the user for information.

### The 'read' Command

Interactive input is handled by the built-in 'read' command. It waits for the user to type something and press Enter, then stores the input in a variable.



## Method 1: The Basic 'read'

A two-step process: prompt, then read.

### How It Works

First, you use 'echo' to print a prompt so the user knows what to enter. Then, you use 'read' followed by a variable name to capture the input.

### Example Code

```
# 1. Print a prompt for the user
echo "Please enter your name:"

# 2. Use `read` to store the input in `user_name`
read user_name

# 3. Use the variable
echo "Hello, $user_name!"
```



## Method 2: 'read' with a Prompt ('-p')

A cleaner, single-line method.

### The '-p' Option

The '-p' (prompt) option lets you combine the 'echo' and 'read' commands into one, which is much more common and efficient.

### Example Code

```
# The prompt is part of the command itself
read -p "What is your favorite color? " user_color

echo "You chose $user_color. A fine choice!"
```

### Best Practice

Using 'read -p' is the preferred method for prompting users for standard input.



## Method 3: Reading Sensitive Data ('-s')

Handling passwords and secret keys.

### The '-s' Option

The '-s' (silent) option prevents the user's typing from being displayed on the screen. The input is still captured by the variable.

### Example Code

```
# -s hides the input, -p provides the prompt
read -s -p "Enter your secret password: " user_password

# It's good practice to echo a blank line afterwards
echo
echo "Password received. Thank you."
```



# Working with Lists: An Introduction to Arrays

## What is an Array?

An array is a special variable that can hold multiple values at once. It's a way to group related pieces of data together.

## Defining an Array

```
# Use parentheses to define an array
# Each value is separated by a space
servers=("web-01" "db-01" "api-server")

# You can also define items with spaces
files=("report.docx" "My Holiday Photos.jpg")
```



# Accessing Array Elements

Retrieving data from an array.

## Accessing a Single Element

Use the index number in square brackets. **Important: The first element is index 0!**

```
#           0           1           2
servers=("web-01" "db-01" "api-server")
echo ${servers[0]} # -> web-01
echo ${servers[2]} # -> api-server
```

## Accessing All Elements

To get all elements at once, use the '@' symbol as the index.

```
1 echo ${servers[@]} # -> web-01 db-01 api-server
```



# Iterating Over an Array

The best way to process each item.

## The 'for' Loop Method

The safest way to loop over an array is to use a 'for' loop with the `${array[@]}` syntax. The quotes are crucial to handle items with spaces correctly.

### Example Code

```
1 files=("report.docx" "My Holiday Photos.jpg")
2
3 for file in "${files[@]"; do
4     echo "Processing file: ' $file '"
5 done
```

Processing file: 'report.docx'

Processing file: 'My Holiday Photos.jpg'



# Iterating Over an Array Indices

The best way to process each item.

## Example Code

```
1 files=("report.docx" "My Holiday Photos.jpg")
2
3 for i in "${!arr[@]}"; do
4     echo "Processing Indices: '$i'"
5 done
```

Processing file: '1'

Processing file: '2'



## Getting the Array Length

Finding out how many elements are in an array.

### The Length Operator (#)

To get the number of elements in an array, place a **#** symbol before the array name in the expansion.

```
1 servers=("web-01" "db-01" "api-server")
2
3 count=${#servers[@]}
4
5 echo "There are $count servers in the list ."
```

### Output

There are 3 servers in the list.



## Example

```
1 # Q1. Print array items joined by , (comma-space).
2 arr=(alpha "beta gamma" delta)
3 ( IFS=' ' ; echo "${arr[*]}" )
4
5 # Q2. Slice / subarray
6 sub="${arr[@]:2:3}" # start=2, len=3
7
8 # Q3. Reverse an array
9 rev=()
10 for ((i=${#arr[@]}-1; i>=0; i--)); { rev+=("${arr[i]}"); }
```



# Organizing Code: An Introduction to Functions

## What is a Function?

A function is a reusable block of code that performs a specific task. They are the key to writing clean, organized, and efficient scripts.

## Why Use Functions?

- **Reusability:** Write code once and call it many times.
- **Readability:** Break down a complex script into logical, named chunks.
- **Maintainability:** A bug in a function only needs to be fixed in one place.



# Defining and Calling a Function

Two common syntaxes.

## Defining a Function

You define a function by giving it a name followed by '()' and curly braces “.

# Syntax 1 (most common and portable)

```
greet() {  
    echo "Hello from the function!"  
}
```

## Calling a Function

To run the code inside the function, you simply type its name.

```
# This will execute the code inside greet()  
greet
```



# Passing Arguments to Functions

Functions have their own set of positional parameters.

## How It Works

Inside a function, `$1`, `$2`,  `$#`, and `$@` refer to the arguments passed *to the function*, not to the main script.

```
1 # This function expects one argument
2 greet_user() {
3     # 'local' prevents the variable from being global
4     local name=$1
5     echo "Welcome, $name!"
6 }
7
8 # Call the function with different arguments
9 greet_user "Pranay"
10 greet_user "Ada"
```

## Output

Welcome, Pranay!  
Welcome, Ada!



# Getting Values From a Function

Two main methods for "returning" data.

## Method 1: Exit Status

Use 'return' to send a number (0-255) indicating success (0) or failure (not 0). Good for conditions.

```
is_root() {  
    [[ $EUID -eq 0 ]] && return 0 || return 1  
}  
  
if is_root; then  
    echo "You are root."  
fi
```

## Method 2: Echo Output

To "return" a string, 'echo' it inside the function and capture it with `$(...)`. This is the standard way.

```
get_timestamp() {  
    date +"%H:%M:%S"  
}  
  
# Capture the output  
now=$(get_timestamp)  
echo "Time is: $now"
```



# Shell Scripting

*Thank You for Listening!*